

2-ma'ruza. Inkapsulyatsiya

Reja:

1. Sinf a'zolariga kirishni boshqarish
2. Yopiq va ochiq kirishni tashkil etish
3. Xossalarni aniqlash
4. Faqat o'qish va yozish xossalari
5. Avtoxossalar va int bloki

Inkapsulyatsiya - bu kodni va boshqarilayotgan ma'lumotlarni birlashtiradigan dasturlash mexanizmi bo'lib, ma'lumotlardan noto'g'ri foydalanish va istisnolarni yo'q qilish uchun ishlatiladi. Obyektga yo'naltirilgan dasturlashda ma'lumotlar va kodlar bitta joyga birlashtirilishi mumkin. Bunday joyda barcha kerakli ma'lumotlar va kodlar mavjud. Kod va ma'lumotlar shu tarzda bir-biriga bog'langanda obyekt yaratiladi. Boshqacha qilib aytganda, obyekt inkapsulyatsiyani qo'llab-quvvatlaydigan elementdir.

Obyektda kodlar va ma'lumotlar ochiq yoki yopiq bo'lishi mumkin. Yopiq ma'lumotlar (yoki kod) ga faqat ruxsat modifikatorlari orqali kirish mumkin. Bu shuni anglatadiki, yopiq ma'lumotlar yoki kodga dasturning obyektidan tashqaridagi qismi kirishi mumkin emas. Agar ma'lumotlar yoki kod ochiq bo'lsa hamda obyekt ichida aniqlangan bo'lsa u, dasturning boshqa qismlari uchun ham ochiq bo'ladi.

C# da inkapsulyatsiyaning asosiy birligi obyekt shaklini belgilaydigan sinfdir. U berilganlarni, shuningdek, ular ustida ishlaydigan kodni tavsiflaydi. C# da sinf e'loni sinfning nusxalari bo'lgan obyektlarni qurish uchun ishlatiladi. Shuning uchun sinf - obyekt qanday qurilganligining sxematik ko'rinishidir.

Sinfni tashkil etuvchi kod va berilganlar sinf a'zolari deb ataladi. Sinf tomonidan aniqlangan berilganlar maydonlar yoki o'zgaruvchilar deb ataladi. Berilganlar asosida ishlaydigan kod bo'lagi metodlar hisoblanadi. C# da metod qism dasturning ekvivalenti bo'lib xizmat qiladi (Boshqa a'zolariga xossalar, hodisalar va konstruktorlar kiradi). Shunday qilib, sinf metodlari sinf tomonidan belgilangan maydonlarga ta'sir qiluvchi kodni o'z ichiga oladi.

Oddiy metodlarga qo'shimcha ravishda, C# xossa deb ataladigan maxsus yordamchi metodlarni taqdim etadi. Ular sinflar va struktura a'zolariga oson kirishni ta'minlaydi, ularning qiymatini o'rganadi yoki ularni o'rnatadi.

Sinf a'zolariga kirishni boshqarish

Sinfidagi inkapsulyatsiya xususiyatini qo'llab-quvvatlash ikkita asosiy afzalliklarni beradi. Birinchidan, sinf berilganlarini kod bilan bog'laydi. Ikkinchidan, sinf o'z a'zolariga kirishni nazorat qilish uchun qulayliklar yaratadi.

C# da asosan ikki turdagi sinf a'zolari mavjud: ochiq va yopiq. Ochiq a'zoga sinfdan tashqarida belgilangan koddan kirish mumkin. Sinfning yopiq a'zosi faqat sinfning o'zida belgilangan metodlar uchun mavjud. Yopiq a'zolar yordamida kirishni nazorat qilish tashkil etiladi.

Sinf a'zolariga kirishni cheklash obyektga yo'naltirilgan dasturlashning asosiy bosqichidir, chunki u obyektidan noto'g'ri foydalanishni oldini olishga imkon beradi. Yopiq ma'lumotlarga faqat belgilangan metodlar orqali kirishga ruxsat beriladi. Yopiq sinf a'zosini sinfdan tashqarida to'g'ridan to'g'ri kodda ishlatib bo'lmaydi. Shu bilan birga, obyektida ma'lumotlar qanday va qachon ishlatilishini to'liq nazorat qilish mumkin.

Kirish modifikatorlari

C# da to'rtta kirish modifikatorlari mavjud: *public*, *private*, *protected* va *internal*. Sinf a'zolariga kirish va ularni boshqarishda *public* va *private* kirish modifikatorlari ishlatiladi. *protected* modifikatori faqat vorislik bilan bog'liq hollarda qo'llaniladi. *internal* modifikatori esa asosan yig'ish uchun mo'ljallangan o'rnatilgan dastur yoki kutubxonani anglatadi.

Agar sinf a'zosi *public* spetsifikatsiyasi bilan belgilangan bo'lsa, unga dasturdagi istalgan boshqa koddan, shu jumladan, boshqa sinflarda belgilangan metodlardan ham kirish mumkin bo'ladi. Agar sinf a'zosi *private* spetsifikatsiyasi bilan belgilangan bo'lsa, unga faqat shu sinfning

boshqa a'zolari kirishi mumkin. Shuning uchun boshqa sinflarning metodlari ushbu sinfning yopiq a'zosiga (private) kirish huquqiga ega emas. agar kirish spetsifikatorlaridan hech biri ko'rsatilmagan bo'lsa, sinf a'zosi yopiq hisoblanadi. Shuning uchun yopiq sinf a'zolarini yaratishda ular uchun *private* spetsifikatsiyasi talab qilinmaydi.

Kirish spetsifikatsiyasi alohida a'zo turi e'lonining qolgan qismidan oldin keladi. Bu shuni anglatadiki, sinf a'zolari e'loni aynan shu bilan boshlanishi kerak. Quyida tegishli misollar keltirilgan.

```
public string errMsg;  
private double bal;  
private bool isError(byte status)  
{  
    // Metod tanasi  
}
```

Ochiq va yopiq modifikatorlar o'rtasidagi farqni tushunish uchun quyidagi dasturini ko'rib chiqamiz.

```
//Sinf a'zolariga ochiq va yopiq kirish o'rtasidagi farqlar.  
using System;  
class MyClass  
{  
    private int alpha; // yopiq a'zo  
    int beta; // standart yopiq a'zo  
    public int gamma; // ochi a'zo  
    // Ushbu sinfning alfa va beta a'zolariga kirish metodlari.  
    public void SetAlpha(int a)  
    {  
        alpha = a;  
    }  
    public int GetAlpha()  
    {  
        return alpha;  
    }  
    public void SetBeta(int a)  
    {  
        beta = a;  
    }  
    public int GetBeta()  
    {  
        return beta;  
    }  
}  
class AccessDemo  
{  
    static void Main()  
    {  
        MyClass ob = new MyClass();  
        // Ushbu sinfning alfa va beta a'zolariga kirish  
        // faqat uning usullari orqali ruxsat etiladi.  
        ob.SetAlpha(-99);  
        ob.SetBeta(19);  
        Console.WriteLine("ob.alpha = " + ob.GetAlpha());  
        Console.WriteLine("ob.beta = " + ob.GetBeta());  
        // Alfa va beta a'zolariga bu sinfda ruxsat berilmaydi.
```

```
// ob.alpha = 10; // Xato! alfa - bu yopiq maydon!  
//ob.beta=9; // Xato! beta - yopiq a'zo!  
// Ushbu sinfnining gamma a'zosiga to'g'ridan to'g'ri kirish mumkin, chunki u  
ochiq.
```

```
ob.gamma = 99;  
Console.WriteLine(ob.gamma);  
Console.ReadKey();
```

```
}  
}
```

Bu yerda, MyClass sinfida alpha a'zo *private* sifatida ko'rsatilgan, beta a'zosi standart bo'yicha *private* bo'ladi va gamma a'zo *public* sifatida belgilanadi. Shunday qilib, alpha va beta a'zolariga berilgan sinfdan boshqa sinfdan to'g'ridan to'g'ri kirish mumkin emas, chunki ular yopiq. Ularga faqat SetAlpha() va GetAlpha() kabi ochiq (public) metodlar orqali kirish mumkin. Agar quyidagi kod satrining izoh belgilarini olib tashlasak:

```
//ob.beta=9; // Xato! beta - yopiq a'zo!
```

yuqoridagi dasturda ruxsat berilmagan amal bajarilishi tufayli kompilyatsiya qilinmaydi.

Xulosa: yopiq a'zo bir xil sinfnining boshqa a'zolari tomonidan erkin foydalanishi mumkin, lekin uning sinfdan tashqarida kodlarga kirish imkoni yo'q.

Yopiq va ochiq kirishni tashkil etish

Yopiq va ochiq foydalanishni to'g'ri tashkil etish obyektga yo'naltirilgan dasturlashda eng muhim tushuncha hisoblanadi. Buning uchun qat'iy qoidalar mavjud bo'lmasa-da, quyida keltirilgan ba'zi umumiy tamoyillar mavjud.

- ✓ Faqat sinfdan ishlatiladigan a'zolar yopiq bo'lishi kerak.
- ✓ Ma'lum bir qiymatlar oralig'iga to'g'ri keladigan ma'lumotlar yopiq bo'lishi kerak va ochiq metodlardan foydalangan holda kirishda sonlarni ko'rsatish oralig'ida tekshirish amalga oshirilishi kerak.
- ✓ Agar a'zoning o'zgarishi sinfdan tashqaridagi a'zolarining o'zgarishiga olib kelsa, ya'ni, obyektning boshqa jihatlariga ta'sir qilsa, a'zo yopiq bo'lishi kerak.
- ✓ Obyektga zarar yetkazishi mumkin bo'lgan a'zolar yopiq bo'lishi kerak. Ushbu a'zolarga ochiq metodlar yordamida kirish kerak, shunda ulardan noto'g'ri foydalanish oldi olinadi.
- ✓ Yopiq ma'lumot qiymatlarini olish va o'rnatish metodlari ochiq bo'lishi kerak.
- ✓ O'zgaruvchilar yopiq bo'lishi uchun hech qanday sabab bo'lmasa, ularni ochiq qilishga ruxsat etiladi.

Xossalarni aniqlash

Standart xossa e'loni quyidagi sintaksisga ega:

```
[modifikator] xossa_turi xossa_nomi  
{  
    get { xossa qiymatini olishda qilinadigan amallar}  
    set { xossa qiymati o'rnatilgandagi amallar}  
}
```

Xossa ta'rifining boshida turli modifikatorlar, xususan, kirish modifikatorlari bo'lishi mumkin. Keyin xossa turi, undan keyin xossa nomi ko'rsatiladi. Xossaning to'liq ko'rinishi ikkita blokdan iborat: *get* va *set*.

get bloki xossa qiymatini olish uchun ishlatiladi. Ushbu blokda *return* kalit so'zidan foydalanib, ba'zi qiymatlar qaytariladi.

set bloki xossa qiymatini belgilaydi. Ushbu blokda *value* parametridan foydalanib, xossaga o'tgan qiymatni olish mumkin.

get va *set* bloklari qo'shimcha yoki yordamchi metodlar, qabul qiluvchi va sozlagich deb ham ataladi.

Ya'ni xossa hech narsani saqlamaydi, u tashqi kod va nom o'zgaruvchisi o'rtasida vositachi vazifasini bajaradi.

Bunga misol sifatida quyidagi dastur kodini ko'rib chiqamiz:

```

using System;
class DemoShaxs {
static void Main() {
    Shaxs shaxs = new Shaxs();
    // Xossani o'rnatish - Set bloki ishlaydi
    // "Temur" qiymati value xossasiga o'tgan qiymatdir
    shaxs.Nom = "Abbos";
    // Biz xossa qiymatini olamiz va uni o'zgaruvchiga tayinlaymiz - Get bloki ishga
    tushiriladi
    string shaxsNom = shaxs.Nom;
    Console.WriteLine(shaxsNom); // Abbas
    Console.ReadKey();
}
}
class Shaxs
{
    private string nom = "Aniqlanmagan";
    public string Nom
    {
        get
        {
            return nom; // xossa qiymatini qaytarish
        }
        set
        {
            nom = value; // yangi xossa qiymatini o'rnatish
        }
    }
}

```

Bu yerda, Shaxs sinfida foydalanuvchi nomini saqlaydigan yopiq nom maydoni aniqlanadi va ochiq Nom xossasi keltirilgan. Ushbu xossa orqali nom o'zgaruvchisiga kirishni boshqarish mumkin. Xossaning *get* blokida maydon qiymatini qaytaramiz:

```

get { return nom; }

```

set blokida nom o'zgaruvchisining qiymatini o'rnatamiz. *value* parametri nom o'zgaruvchisiga uzatiladigan qiymatni ifodalaydi.

```

set { nom = value; }

```

Dasturda ushbu xossaga oddiy maydon kabi kiriladi. Agar unga qiymat berilsa, u holda *set* bloki ishga tushadi va qiymat *value* parametriga o'tkaziladi:

```

shaxs.Nom = "Abbos";

```

Agar xossa qiymati olinsa, *get* bloki ishga tushiriladi, bu, asosan, nom o'zgaruvchisining qiymatini qaytaradi:

```

string shaxsNom = shaxs.Nom;

```

Agar ushbu holatda oddiy sinf maydonlari bilan ham ishlash mumkin bo'lsa, xossalar nima uchun kerak? degan savol tug'ilishi tabiiy. Ammo xossalar qiymatni o'rnatish yoki olishda kerak bo'lishi mumkin. Masalan, yoshni tekshiradigan xossani ifodalaydigan dastur kodini yozamiz:

```

using System;
class DemoShaxs {
static void Main() {
    Shaxs shaxs = new Shaxs();
    Console.WriteLine(shaxs.Yosh); // 1
    // xossa qiymatini o'zgartirish
    shaxs.Yosh = 32;
}
}

```

```

        Console.WriteLine(shaxs.Yosh); // 32
        // yaroqsiz qiymatni o'tkazishga urinish
        shaxs.Yosh = -23; // Yosh 1 dan 120 gacha bo'lishi kerak
        Console.WriteLine(shaxs.Yosh); // 32 - yosh o'zgarmadi
        Console.ReadKey();
    }
}
class Shaxs
{
    int yosh = 1;
    public int Yosh
    {
        set
        {
            if (value < 1 || value > 120)
                Console.WriteLine("Yosh 1 dan 120 gacha bo'lishi kerak");
            else
                yosh = value;
        }
        get { return yosh; }
    }
}

```

Bunday holda, yosh o'zgaruvchisi foydalanuvchining yoshini saqlaydi. Bu o'zgaruvchiga to'g'ridan to'g'ri kira olmaymiz, faqat Yosh xossasi orqali kirish mumkin. Bundan tashqari *set* blokda agar u berilgan diapazonga mos kelsa, qiymatni o'rnatamiz. Agar Yosh xossasiga ushbu oraliqda bo'lmagan qiymat o'tkazilsa, o'zgaruvchining qiymati o'zgarmaydi:

```

1
shaxs.Yosh = -23;
Dasturning natijasi quyidagicha bo'ladi:

```

```

1
32
Yosh 1 dan 120 gacha bo'lishi kerak
32

```

Shunday qilib, xossa - vositachilik qilish va obyekt ma'lumotlariga kirishni boshqarish imkonini beradi.

Faqat o'qish va yozish xossalari

set va *get* bloklari bir vaqtning o'zida xossada bo'lishi shart emas. Agar xossa faqat *get* blokini aniqlasa, u holda bu xossa faqat o'qish uchun mo'ljallangan, uning qiymati olinishi mumkin, lekin u o'rnatilmaydi.

Agar xossa faqat *set* blokiga ega bo'lsa, u holda bu xossa faqat yozish uchun mo'ljallangan, faqat qiymatni belgilash mumkin, lekin uning qiymati olinmaydi.

```

using System;
class DemoShaxs {
static void Main() {
    Shaxs shaxs = new Shaxs();
    // o'qish uchun xossa - faqat qiymatni olish mumkin
    Console.WriteLine(shaxs.Nom); // Temur
    // o'rnatib bo'lmaydi
    // shaxs.Nom = "Bobur"; // !Xato
    // yozish uchun xossa- faqat qiymatni belgilash mumkin
    shaxs.Yosh = 32;
    // lekin olib bo'lmaydi
}
}

```

```

        // Console.WriteLine(shaxs.Yosh); // !Xato
        shaxs.Print();
        Console.ReadKey();
    }
}
class Shaxs
{
    string nom = "Tolmas";
    int yosh = 1;
    // faqat yozish xossasi
    public int Yosh
    {
        set { yosh = value; }
    }
    // faqat o'qish xossasi
    public string Nom
    {
        get { return nom; }
    }
    public void Print() => Console.WriteLine($"Nom: {nom} Yosh: {yosh}");
}

```

Bu yerda, Nom xossasi faqat o'qish uchun mo'ljallangan, chunki unda faqat *get* bloki mavjud:

```

public string Nom
{
    get { return nom; }
}

```

Uning qiymatini olish mumkin, lekin qiymat berib bo'lmaydi:

```

Console.WriteLine(shaxs.Yosh); // olish mumkin
shaxs.Yosh = "Abbos"; // !Xato - o'rnatib bo'lmaydi

```

Boshqa tomondan, Yosh xossasi faqat yozish uchun mo'ljallangan, chunki u faqat *set* blokka ega:

```

public int Yosh
{
    set { yosh = value; }
}

```

Uning qiymatini belgilash mumkin, lekin uni olib bo'lmaydi:

```

shaxs.Yosh = 32; // qiymat berish mumkin
Console.WriteLine(shaxs.Yosh); // !Xato - qiymatni olish mumkin emas

```

Hisoblangan xossalar.

Xossalar ixtiyoriy ravishda ma'lum bir o'zgaruvchi bilan bog'lanadi. Ular turli xil ifodalar asosida hisoblanishi mumkin:

```

using System;
class DemoShaxs
{
    static void Main()
    {
        Shaxs shaxs = new Shaxs("Nodir", "Soatov");
        Console.WriteLine(shaxs.Nom); // Nodir Soatov
        Console.ReadKey();
    }
}
class Shaxs

```

```

{
    string ism;
    string familiya;
    public string Nom
    {
        get { return $"{ism} {familiya}"; }
    }
    public Shaxs(string ism, string familiya)
    {
        this.ism = ism;
        this.familiya = familiya;
    }
}

```

Bunday holda, Shaxs sinfida ism va familiya o'zgaruvchilari qiymatlari asosida umumiy qiymatni qaytaruvchi faqat o'qish uchun mo'ljallangan Nom xossasi mavjud.

Kirish modifikatorlari

Kirish modifikatorlarini nafaqat barcha xossaga, balki alohida *get* va *set* bloklariga ham qo'llashimiz mumkin:

```

using System;
class DemoShaxs {
    static void Main() {
        Shaxs tom = new Shaxs("Nodir");
        // Xato - set private modifikatori bilan e'lon qilingan
        //tom.Nom = "Nurbek";
        Console.WriteLine(tom.Nom); // Nodir
        Console.ReadKey();
    }
}
class Shaxs
{
    string nom = "";
    public string Nom
    {
        get { return nom; }
        private set { nom = value; }
    }
    public Shaxs(string nom) => Nom = nom;
}

```

Endi yopiq *set* blokini faqat shu sinfda - uning metodlari, xossalari, konstruktorida ishlatish mumkin, lekin boshqa sinfda emas:

Xossalarda modifikatorlardan foydalanishda bir qator cheklovlarni yodda tutish kerak:

- Agar xossa ikkala blokga (*set* va *get*) ega bo'lsa, *set* yoki *get* bloki uchun modifikator o'rnatilishi mumkin.
- Faqat bitta *set* yoki *get* bloki kirish modifikatoriga ega bo'lishi mumkin, ikkalasiga bir vaqtda mumkin emas.
- *set* yoki *get* blokiga kirish modifikatori xossaga kirish modifikatoridan ko'ra ko'proq cheklov bo'lishi kerak. Misol uchun, agar xossa umumiy modifikatorga ega bo'lsa, *set/get* blokida faqat *protected internal*, *internal*, *protected*, *private protected* va *private* modifikatorlari bo'lishi mumkin.

Avttoxossalar

Xossalar sinf maydonlariga kirishni nazorat qiladi. Agar ko'proq maydonlar bo'lsa, unda har bir maydonni aniqlash va unga bir xil turdagi xossani yozish zerikarli bo'ladi. Shuning uchun .NET ga avtomatik xossalar qo'shildi. Ularning qisqartmasi quyidagicha:

```
class Shaxs
{
    public string Nom { get; set; }
    public int Yosh { get; set; }

    public Shaxs(string nom, int yosh)
    {
        Nom = nom;
        Yosh = yosh;
    }
}
```

Bu yerda, xossalar uchun maydonlar ham yaratilgan, faqat ular koddagi dasturchi tomonidan emas, balki kompilyatsiya paytida avtomatik ravishda kompilyator tomonidan yaratiladi.

Avtoxossalarning afzalligi: agar ular mazmunan faqat avtomatik yaratilgan o'zgaruvchiga ishora qilsa, nega avtoxossalarsiz o'zgaruvchiga bevosita murojaat qilmaslik kerak? Chunki istalgan vaqtda, avtomatik xossani oddiy xossaga kengaytira olamiz, unga qandaydir o'ziga xos mantiq qo'shamiz.

Shuni esda tutish zarurki, standart xossalarga ega bo'lganidek, faqat yozish uchun avtomatik xossa yaratib bo'lmaydi.

Avtoxossalarga standart qiymatlar berilishi mumkin (avtoxossani ishga tushirish):

```
using System;
class DemoShaxs {
    static void Main() {
        Shaxs tom = new Shaxs();
        Console.WriteLine(tom.Nom); // Abbos
        Console.WriteLine(tom.Yosh); // 32
        Console.ReadKey();
    }
}
class Shaxs
{
    public string Nom { get; set; } = "Abbos";
    public int Yosh { get; set; } = 32;
}
```

Agar biz Shaxs obyekti uchun Nom va Yosh xossalarining qiymatlarini ko'rsatmasak, standart qiymatlar qo'llaniladi.

Avtoxossalarda kirish modifikatorlari ham bo'lishi mumkin:

```
class Shaxs
{
    public string Nom { private set; get; }
    public Shaxs(string nom) => Nom = nom;
}
```

set blokni olib tashlash va avtoxossani faqat o'qish uchun e'lon qilish mumkin. Bunday holda, ushbu xossaning qiymatini saqlash uchun, faqat o'qiladigan modifikatorga ega bo'lgan maydon bevosita yaratiladi, shuning uchun bunday get-xossalar misoldagi kabi sinf konstruktorida ham o'rnatilishi mumkin. Xossa ishga tushirilganda:

```
class Shaxs
{
```

```
// xossani ishga tushirish orqali
public string Nom { get; } = "Abbos";
// konstruktor orqali
public Shaxs(string nom) => Nom = nom;
}
```

init bloki

C# ning 9.0 versiyasidan boshlab, xossalardagi sozlagichlarni *init* kalit soʻzi yordamida aniqlash mumkin ("initializatsiya" soʻzidan olingan boʻlib - *init* bloki xossani ishga tushirish uchun moʻljallangan). *init* bilan xossa qiymatlarini oʻrnatish uchun faqat initsializator yoki konstruktordan foydalanish mumkin yoki uni eʼlon qilganda uning qiymatini belgilash mumkin. Bunday xossalarning qiymatlari ishga tushirilgandan soʻng, ularning qiymatlarini oʻzgartirib boʻlmaydi yaʼni faqat oʻqish uchun moʻljallangan. Shu nuqtayi nazardan, *init* xossalari oʻqish xossasiga yaqinroqdir. Farqi shundaki, initsializatorda *init* xossalarini ham berish mumkin (initsializatorda xossalarni oʻrnatib boʻlmaydi):

```
Shaxs shaxs = new Shaxs();
//shaxs.Nom = "Nurbek";
//!Xato - ishga tushirilgach qiymatni oʻzgartirib boʻlmaydi
Console.WriteLine(shaxs.Nom); // Aniqlanmagan
public class Shaxs
{
    public string Nom { get; init; } = "Aniqlanmagan";
}
```

Bu holda Shaxs sinfi Nom xossasi uchun *set* oʻrniga *init* operatoridan foydalanadi. Natijada,

```
Shaxs shaxs = new Shaxs();
```

obyektni uning barcha xossalarini ishga tushirgan holda yaratishi kerak. Bu holda Nom xossasi oʻz qiymati sifatida "Aniqlanmagan" satrini chiqaradi. Biroq xossa ishga tushirilganligi sababli:

```
shaxs.Nom = "Nurbek"; //!Xato
```

ushbu satr xato hisoblanadi.

Bunday xossani qoʻllash uchun ikkinchi yoʻl - konstruktordan foydalaniladi:

```
Shaxs shaxs = new("Jasur");
Console.WriteLine(shaxs.Nom); // Jasur
public class Shaxs
{
    public Shaxs(string nom) => Nom = nom;
    public string Nom { get; init; }
}
```

Uchinchi yoʻl - initsializator orqali:

```
Shaxs shaxs = new Shaxs() { Nom = "Bobur" };
Console.WriteLine(shaxs.Nom); // Bobur
public class Shaxs
{
    public string Nom { get; init; } = "";
}
```

Keyingi usul - *init* modifikatori bilan boshqa xossa orqali sozlash:

```
var shaxs = new Shaxs() { Nom = "Sherzod" };
Console.WriteLine(shaxs.Nom); // Sherzod
Console.WriteLine(shaxs.Email); // Sherzod@gmail.com
public class Shaxs
{
    string nom = "";
    public string Nom
```

```

    {
        get { return nom; }
        init
        {
            nom = value;
            Email = "${value}@gmail.com";
        }
    }
    public string Email { get; init; } = "";
}

```

Bunday holda, *init* xossasida Nom maydon nomini boshqaruvchi to‘liq xossaga kengayadi. Bundan tashqari, *init* ifodasida yana bir Email *init* xossa o‘rnatiladi. Qiymatni o‘rnatish uchun Nom xossasi qiymatidan foydalaniladi, Nom xossasidan elektron pochta manzilining qiymati olinadi.

Agar xossani e‘lon qilishda qiymat ko‘rsatilgan bo‘lsa, uni konstruktorda o‘zgartirish mumkin. Konstruktorda o‘rnatilgan qiymatni initsializatorida o‘zgartirish mumkin. Lekin, ishga tushirish jarayoni tugaydi va qiymatni o‘zgartirib bo‘lmaydi.

Xossalarning qisqartirilgan ko‘rinishi.

Metodlar singari, xossalarni ham qisqartirish mumkin. *get* va *set* bloklari oddiy metodlar kabi maxsus metodlarni ifodalaganligi sababli, ular bitta ifodadan iborat bo‘lsa, *=>* operatori yordamida qisqartirish mumkin:

```

class Shaxs
{
    string nom;
    public string Nom
    {
        get => nom;
        set => nom = value;
    }
}

```

Bundan tashqari, xossani bir butun sifatida qisqartirish mumkin:

```

class Shaxs
{
    string nom;
    // "public string Nom { get { return nom; } }" bilan ekvivalent
    public string Nom => nom;
}

```